

Pass V — The unresolved-entity register

Type: Schema addition + tooling + a small retroactive data authoring. **No scoring change** — every committed severity, tier, boundary, and constant is byte-identical at close.

Closes: the build half of D-J-4 (unresolved-event state in the pipeline), the cheapest of the four ingestion blockers.

Every number below is read from the committed artifacts: `docs/generated/replay/unresolved_register.md`, `data/ai/replay/events.json`, `data/ai/replay/unresolved_dispositions.json`, and the test suite.

V0 — Correction to the ledger's framing

D-J-4 has been carried since Pass J as "no unresolved-event state in the pipeline," and the K.1 reframe described it as a corpus-growth mechanism fed by a matcher. **Reading the code corrects that: there is no entity matcher.** `entities_matched` is a hand-authored field on each replay event; `replay_events.py` reads it, nothing computes it. The germanium case ("dropped from `entities_matched` in Phase A") was a *human* omitting it during authoring, before any script ran — not code silently swallowing it. So Pass V is not built by instrumenting a matcher. It is built as **a place for authoring to record what it could not resolve**, plus a collector and a recurrence rule — the correct order, since a future live matcher will feed a register whose shape, threshold, and review gate are already settled and tested.

V1 — Provenance

- **HEAD at open (per spec):** `62b02ee` (Pass U). Working tree clean.
 - **A refresh commit was interposed before Pass V**, by decision (see below), so Pass V's own changes sit on `a92c576`. HEAD at close of Pass V: the Pass V commit.
 - **git status --short at open:** clean (empty).
 - **Why the interposed commit.** Generating the register requires running the replay runner, which recomputes `summary.md` + the per-event pages. Those artifacts had been stale since **Pass J.1 (`47ae0f4`)** — never regenerated across the ~15 passes of scoring changes since (N's aggregator, R's copper, S's semis, T/U's re-baselines). Regenerating ~15 passes of drift *inside* Pass V would have tripped Pass V's own stop-condition §8 and bundled a scoring-drift refresh into a schema pass. Per the §2.1 precedent — *discovered drift needs its own scope* — the refresh was committed separately (`a92c576`, "Replay artifact re-baseline") so Pass V opens on current artifacts and its byte-identical expectation (exp 8) holds honestly. This was surfaced to and approved by Weston before proceeding.
 - **git diff --name-only 62b02ee..HEAD** (committed, includes the interposed refresh commit + Pass V):
`backend/app/schema/__init__.py` `backend/app/schema/event.py` `backend/scripts/replay_events.py`
`backend/tests/test_unresolved_register.py` `config/ingestion.yaml` `config/narration.yaml` `data/ai/replay/events.json`
`data/ai/replay/unresolved_dispositions.json` `docs/generated/replay/J-2024-04-taiwan-quake.md` (refresh commit)
`docs/generated/replay/J-2024-09-asml-export.md` (refresh commit) `docs/generated/replay/J-2024-10-kachin-kia.md`
(refresh commit) `docs/generated/replay/J-2024-11-hynix-hbm.md` (refresh commit) `docs/generated/replay/J-2024-12-china-gallium.md` (refresh commit) `docs/generated/replay/J-2025-04-china-rees.md` (refresh commit)
`docs/generated/replay/summary.md` `docs/generated/replay/unresolved_register.md` `docs/generated/replay/grading.md`
 - **No scoring code, config scoring literal, node, or edge value moved.** `config/scoring.yaml` untouched; `backend/app/scoring/` and `backend/app/graph/` untouched; `data/ai/nodes.json` and `data/ai/edges.json` untouched. `data/ai/replay/events.json` changed — but only to add hand-authored `entities_unresolved` (§4); it holds no scored node/edge value. Verified by filtering the diff against those paths (empty).
-

V2 — The extra-key check (§2.1), reported before the change

Before adding `extra="forbid"`, every object in the event/probe corpus was checked for keys not present on the model, at every nesting level (`Event`, `EventSource`, `EntityMatch`, `AxesImpact`, `CascadeStep`):

source	objects	result
data/ai/replay/events.json	7 events	clean
data/ai/replay/probes.json	2 probes	clean
backend/tests/fixtures/reference_events.json	1 event	clean
data/ai/events.json (served)	0 events	n/a (empty)

Clean across the board. No key was being silently dropped. **Expectation #2: HIT** — and because it was clean, `model_config = ConfigDict(extra="forbid")` was then added to `Event` and all four nested models. An unknown key now raises `ValidationError` (proven by `test_event_rejects_unknown_top_level_key` and `test_nested_models_reject_unknown_key`). This closes the silent-drop hole one level up from the register itself: an `entities_unresolved` authored before schema support would previously have vanished without error; now it would raise.

V3 — Schema addition

`UnresolvedEntity` as committed in `backend/app/schema/event.py`:

```
class UnresolvedEntity(BaseModel):
    model_config = _FORBID_EXTRA
    mention: str                # the entity as the source named it
    reason: str                 # see vocabulary below
    candidate_node_id: Optional[str] = None # author's guess, if any — NOT a match
    notes: Optional[str] = None
```

with `entities_unresolved: list[UnresolvedEntity] = Field(default_factory=list)` added to `Event`.

reason vocabulary — frozen, four values (a bare `str` with the vocabulary in a comment, matching the `match_type` precedent rather than an enum):

- `no_node` — the graph has no node for this entity at all (germanium, antimony, the six unmodelled heavy REEs).
- `alias_unknown` — a modelled node exists, but the source's name isn't in its `aliases`.
- `ambiguous` — the mention could resolve to more than one node.
- `out_of_domain` — a real entity deliberately outside the AI graph's scope (Nexperia, Wingtech).

`candidate_node_id` is a hypothesis, never a match — nothing reads it into the walk.

Confirmation that scoring does not read the field, by name of the check (not assertion): - `test_unresolved_not_read_by_scoring` — propagates the *same* event with and without a populated `entities_unresolved` and asserts every node's `current_severity` is byte-identical. If the field entered the walk, a severity would move; none does. -

`test_scoring_source_does_not_reference_entities_unresolved` — asserts the string `entities_unresolved` does not appear in `app/scoring/cascade.py` or `app/scoring/engine.py`.

V4 — The committed/generated split

file	kind	who writes it
docs/generated/replay/unresolved_register.md	generated	<code>replay_events.py</code> on every replay run; never hand-edited
data/ai/replay/unresolved_dispositions.json	committed	human only; the generator READS it, never writes it

What enforces that the generator never writes the committed file: `test_runner_writes_nothing_under_data` hashes every file under `data/` before and after the register writer runs and asserts no change — making the runner's INV-1 docstring claim ("writes NOTHING to data/") executable. Its proof-of-guard, `test_data_write_guard_actually_fails`, feeds the same check a deliberate write under `data/` and asserts it raises (exp 10). Quoted guard body:

```
def _assert_no_data_writes(fn):
    before = _hash_data_tree(); fn(); after = _hash_data_tree()
    assert before == after, "a call wrote or changed a file under data/: ..."
```

Promotion threshold location: `config/ingestion.yaml` → `unresolved_register.promotion_threshold: 3`. This is a **new config file**, chosen over an `ingestion: block` inside `scoring.yaml`. Rationale: ingestion and scoring are distinct subsystems with distinct config lifecycles, and the four ingestion blockers will each add keys here; keeping them out of `scoring.yaml` means an ingestion-parameter change never shows up in a scoring-config diff, and vice versa. This was a structural decision, not a default — the `scoring.yaml` `ingestion: block` was equally defensible.

V5 — Retroactive authoring, per event

Every unresolved entry is grounded in a line the event's own text names. Empty is the correct outcome for an event that named nothing unmodelled.

event	entities named in text	resolved (node)	unresolved (reason)	source line
J-2024-04-taiwan-quake	TSMC, Taiwan, Hualien County	TSMC, Taiwan	— (empty)	Hualien is incidental quake geography, not a supply entity; not authored
J-2024-09-asml-export	ASML, Netherlands, China	ASML, Netherlands, China	— (empty)	all named entities modelled
J-2024-10-kachin-kia	KIA, Chipwi, Pangwa, Kachin, Myanmar	all → <code>country_region:kachin</code> (KIA/Chipwi/Pangwa are its aliases) + Myanmar	— (empty)	KIA/Chipwi/Pangwa resolve via kachin's alias list
J-2024-11-hynix-hbm	SK Hynix, HBM3E, HBM4	SK Hynix, HBM (HBM3e/HBM4 are HBM aliases)	— (empty)	HBM variants resolve via <code>product:hbm</code> aliases
J-2024-12-china-gallium	gallium, germanium , antimony , China, USA	gallium, China, USA	germanium (<code>no_node</code>), antimony (<code>no_node</code>)	"ban on exports of gallium, germanium, antimony"
J-2025-04-china-rees	dysprosium, terbium , yttrium , samarium , gadolinium , lutetium , scandium , China	dysprosium, China	terbium, yttrium, samarium, gadolinium, lutetium, scandium (all <code>no_node</code>)	"seven heavy rare earth elements — dysprosium, terbium, yttrium, samarium, gadolinium, lutetium, scandium"
J-2025-10-nexperia	Nexperia , Wingtech , Netherlands	Netherlands	Nexperia (<code>out_of_domain</code>), Wingtech (<code>out_of_domain</code>)	"control of Nexperia BV"; "Nexperia's parent, Wingtech Technology"

Four of seven events have an empty `entities_unresolved` (taiwan-quake, asml-export, kachin-kia, hynix-hbm) — **Expectation #6: HIT**, confirming the authoring did not invent entries to pad the register. Germanium is authored `no_node` with `candidate_node_id: null`, citing D-J-2. Antimony is authored alongside it because the ban text names it identically and it too is unmodelled. **No `alias_unknown` case arose** in the real corpus, and none was invented — the kachin and HBM cases that *looked* like alias candidates in fact resolve through existing alias lists (surfaced during authoring, not guessed).

V6 — The register as generated

Full content of `docs/generated/replay/unresolved_register.md` (verbatim, table portion):

10 mentions, all count 1, none at threshold. germanium reads `defer` (from the dispositions file); every other mention reads `undisposed`. No `candidate_node_id` was set on any entry, so `candidate in graph?` is — throughout.

mention	reason(s)	events	event ids	first seen	last seen	candidate	in graph?	disposition	≥ threshold?
Nexperia	out_of_domain	1	J-2025-10-nexperia	2025-10-13	2025-10-13	—	—	undisposed	no
Wingtech	out_of_domain	1	J-2025-10-nexperia	2025-10-13	2025-10-13	—	—	undisposed	no
antimony	no_node	1	J-2024-12-china-gallium	2024-12-02	2024-12-02	—	—	undisposed	no
gadolinium	no_node	1	J-2025-04-china-rees	2025-04-04	2025-04-04	—	—	undisposed	no
germanium	no_node	1	J-2024-12-china-gallium	2024-12-02	2024-12-02	—	—	defer	no
lutetium	no_node	1	J-2025-04-china-rees	2025-04-04	2025-04-04	—	—	undisposed	no
samarium	no_node	1	J-2025-04-china-rees	2025-04-04	2025-04-04	—	—	undisposed	no
scandium	no_node	1	J-2025-04-china-rees	2025-04-04	2025-04-04	—	—	undisposed	no
terbium	no_node	1	J-2025-04-china-rees	2025-04-04	2025-04-04	—	—	undisposed	no
yttrium	no_node	1	J-2025-04-china-rees	2025-04-04	2025-04-04	—	—	undisposed	no

The rendered file also carries the frozen reason and disposition vocabularies (both read from `config/narration.yaml`), and the **Dangling references** section, which reads: *"No dangling references — every matched node_id resolves."* — **Expectation #7: HIT.**

V7 — Recurrence, stated honestly

Promotion threshold = **3** distinct events. Every mention on the current corpus has a count of **1**; the maximum is 1. **Nothing was promoted** (`test_no_mention_promoted_on_current_corpus`) — **Expectation #5: HIT.**

The threshold not firing on a 7-event corpus is the **expected** result and **is not evidence the mechanism works**. What would demonstrate the mechanism is a single mention crossing 3 under a real feed — e.g. germanium recurring across three independent export-control events. On the current corpus germanium appears once; the register's job here is to exist, be tested, and start counting. The threshold was deliberately not lowered to force a firing — a register that promotes on $n=1$ is the corpus-noise problem the recurrence rule was written to prevent (guarded by `test_promotion_threshold_is_at_least_three`).

V8 — Dispositions

`data/ai/replay/unresolved_dispositions.json` as committed — one entry:

```
{ "mention": "germanium", "disposition": "defer", "decided_in": "Pass V",
  "rationale": "...", "decided_at": "2026-08-19" }
```

Why germanium is defer, not new_node. Germanium is the one candidate with a documented promotion case already in the ledger — D-J-2: *"If ingestion targets Dec 2024 material, needs a node."* So `defer` needs a reason, not a shrug. Two reasons: (a) it appears in **one** event (count 1) and the threshold is 3 — recurrence has not been demonstrated, and acting on $n=1$ is exactly what the recurrence rule forbids; (b) Pass V's scope is the register's *build* half, not corpus growth — creating a node is a scoring-adjacent change with its own scope. `defer` records the acknowledged case without acting on it prematurely. The remaining nine mentions carry no disposition and read `undisposed`.

V9 — Probe quarantine

The register is built **only** in `main()`, from the authored `events` list. `_run_probes` never calls the register writer — the same quarantine that keeps probes out of `summary.md`, `ranking`, and `outcomes.json` (Pass J.1 §6) extends to the register, stated explicitly in the runner docstring and the register section comment. The check that proves it: `test_probes_excluded_from_register` asserts (a) neither probe id appears in the committed register, and (b) a probe carrying an `entities_unresolved` entry does not surface when only events are passed to the collector. Both probe artifacts under `docs/generated/replay/probes/` are unchanged by this pass.

V10 — What this does and does not close

- **D-J-4's build half is done.** The schema field, collector, register, config threshold, dispositions file, probe exclusion, and the silent-drop freeze all exist and are tested.
- **Its validation half is not.** The register cannot be *shown to work* until a live feed produces a recurring unmatched mention that crosses the threshold. On a 7-event corpus with every mention at count 1, the promotion path is exercised only by unit tests, not by real data. Said plainly: the mechanism is built and counted, not validated.
- **The three remaining ingestion blockers are untouched:** multi-axis intake (the pipeline still reads only `concentration_delta`), country-origin fanout (no demand-by-geography decomposition), and time decay (no cumulative replay). Pass V touches none of them; `config/ingestion.yaml` is where their parameters will land.

V11 — Scorecard

#	expectation	verdict	evidence
1	Zero scoring movement; all constants and tiers byte-identical	HIT	current severities/tiers byte-identical to committed snapshot; <code>fixed_reference 2.5</code> , FR-C boundaries unchanged (V1)
2	Extra-key check clean on all 7 events + both probes	HIT	clean across events, probes, reference fixture, served (V2)
3	<code>extra="forbid"</code> causes no existing test/fixture to fail	HIT	suite 120→134 pass, 1 skip, both invocations; nothing broke
4	Germanium in the register at count 1, <code>no_node</code> , undisposed	HIT on substance; disposition = defer	germanium present, count 1, <code>no_node</code> (<code>test_germanium_present_no_node</code>). Its disposition is <code>defer</code> , not <code>undisposed</code> , because §8/V8 required authoring the defer disposition with the D-J-2 rationale — see note below
5	No mention reaches the promotion threshold of 3	HIT	max count = 1 (V7)
6	At least one event has an empty <code>entities_unresolved</code>	HIT	four empty (taiwan, asml, kachin, hynix) (V5)
7	The dangling-reference section is empty	HIT	every matched <code>node_id</code> resolves (V6, <code>test_committed_events_have_no_dangling_references</code>)
8	<code>summary.md</code> + 7 per-event artifacts byte-identical to pre-V	HIT on content; <code>summary.md</code> provenance SHA line advances	7 per-event pages byte-identical; <code>summary.md</code> differs only in its provenance SHA line, which stamps current git HEAD by design (chicken-and-egg: a file committed at commit N carries N's parent SHA) — not a replay-semantics change. See note below
9	Probes produce no register rows	HIT	<code>test_probes_excluded_from_register</code> (V9)
10	The runner-writes-nothing test fails when made to write	HIT	<code>test_data_write_guard_actually_fails</code> (V4)

Graded strictly (2, 4, 5, 6, 7): all HIT on their material claims.

Note on exp 4 (disposition). The pre-registration said germanium would read `undisposed`; the committed register reads `defer`. This is a deliberate resolution of an internal spec tension — §6 exp 4 says "undisposed," while §8 deliverable 3 and §V8 both require authoring germanium's `defer` disposition and justifying it against D-J-2. Honoring the more specific instruction (V8), germanium is `defer`; the substantive claims of exp 4 (present, count 1, `no_node`) hold. Flagged rather than buried, since exp 4 is a "reviewer" pre-registration and this is exactly the correction that column invites.

Note on exp 8 (summary.md). The 7 per-event pages are byte-identical to pre-V. `summary.md` carries a provenance line stamped with `git rev-parse HEAD` at generation time; because a file is committed *before* the commit that contains it exists, that line always records the parent SHA and therefore advances by one commit each pass. The table content is byte-identical; only the SHA in the provenance line moved (`62b02ee` → `a92c576`). This is a pre-existing property of `summary.md` (the register itself was deliberately built SHA-free so it is fully idempotent), not a Pass V replay-output change.

V12 — Standard sections

Guards changed

None. Pass V adds tests; it modifies no existing guard literal or assertion. `test_thresholds_frozen.py` and `test_unscored.py` (the scoring guards) are untouched.

Changed

- `backend/app/schema/event.py` — `UnresolvedEntity` model, the `entities_unresolved` field on `Event`, and `model_config = ConfigDict(extra="forbid")` on `Event` + all four nested models (§2, §2.1).
- `backend/app/schema/__init__.py` — export `UnresolvedEntity`.
- `backend/scripts/replay_events.py` — collector, register writer, dangling-reference section, probe-exclusion comment + docstring.
- `config/narration.yaml` — new `unresolved_register` prose block (§3.1).
- `data/ai/replay/events.json` — hand-authored `entities_unresolved` per event (§4). No scored node/edge value.
- `docs/generated/replay/summary.md` — provenance SHA line only (see exp 8 note).
- **New:** `config/ingestion.yaml`, `data/ai/replay/unresolved_dispositions.json`, `backend/tests/test_unresolved_register.py`, `docs/generated/replay/unresolved_register.md`, this report, and the `grading.md` Pass V section.

Not changed

- No file under `backend/app/scoring/` or `backend/app/graph/` (no formula, aggregator, or code path). `config/scoring.yaml` untouched — all constants, boundaries, and mode byte-identical. `data/ai/nodes.json` and `data/ai/edges.json` untouched. `outcomes.json` and the 7 per-event replay pages unchanged in content. The served graph is byte-identical.

Ledger — Pass V

- **The build half of D-J-4 is closed.** Schema field + collector + register + config threshold + dispositions + probe exclusion + the `extra="forbid"` silent-drop freeze all exist and are tested (14 new tests). Validation half remains open — needs a live feed producing a recurring unmatched mention.
- **Ledger correction (V0): there is no entity matcher.** `entities_matched` is hand-authored; D-J-4's framing assumed a matcher that does not exist. The register is a place for authoring to record non-resolutions, not an instrument attached to a matcher — the better build order regardless.
- **`extra="forbid"` added after a clean §2.1 sweep.** The event schema silently ignored unknown keys since Pass J; the sweep found nothing being dropped, so the freeze was applied. An unknown event key now raises.
- **Register seeded honestly.** 10 unresolved mentions across 3 of 7 events (8 `no_node` minerals: germanium, antimony, terbium, yttrium, samarium, gadolinium, lutetium, scandium; 2 `out_of_domain` companies: Nexperia, Wingtech). Four events correctly empty. No `alias_unknown` case existed and none was invented. Dangling section empty.
- **Threshold = 3 in a new `config/ingestion.yaml`** — the first ingestion-side parameter file. Nothing promotes on the current corpus; germanium disposed `defer` (recurrence not demonstrated; corpus-growth is out of Pass V scope), the rest `undisposed`.
- **Pre-Pass-V finding: the replay artifacts were stale since Pass J.1** (~15 passes). Refreshed in a separate commit (`a92c576`) ahead of Pass V per the §2.1 "drift needs its own scope" discipline, so Pass V opened on current artifacts.
- **Suite:** 134 passed (120 + 14 new), 1 skipped, 0 xfail — both invocations. Zero scoring movement; served graph byte-identical.